



PACCIOLI

"Desarrolla tu potencial, transforma tu mundo"

INSTITUTO TECNOLÓGICO "PACCIOLI"
CARRERA DE SISTEMAS INFORMATICOS

**SISTEMA INTELIGENTE DE PREDICCIÓN DEL RENDIMIENTO
ACADÉMICO MEDIANTE EL ANÁLISIS DE
HÁBITOS DE ESTUDIO**

Postulante:

Roy Paules Rosa

Tutor:

Tec. Sup. Kevin Camacho Rodriguez

Punata – Cochabamba -Bolivia

2026

ÍNDICE

1. INVESTIGACION PRELIMINAR.....	1
1.1. ACLARACIÓN DE LA SOLICITUD	1
1.2. ESTUDIO DE FACTIBILIDAD	3
1.2.1 Factibilidad Técnica:.....	3
1.2.2 Factibilidad Operativa:.....	4
1.2.3 Factibilidad Económica:.....	5
1.2.4 Riesgos identificados y mitigación:.....	6
2. DETERMINACIÓN DE REQUERIMIENTOS	7
2.1. RECOPIACIÓN DE INFORMACIÓN	7
2.1.1. Técnicas Utilizadas:.....	8
2.1.2. Fuentes de Información Consultadas:	9
2.2. ANÁLISIS DE LA INFORMACIÓN.....	10
2.2.1 Clasificación de Requerimientos:.....	11
2.2.1.1. Requerimientos Funcionales:	11
2.2.1.2. Requerimientos No Funcionales:.....	12
2.2.2. Priorización de Requerimientos (MoSCoW):.....	14
2.2.3. Análisis de Consistencia y Completitud:	15
2.2.4. Matriz de Trazabilidad Requirimientos-Casos de Uso:	16
3. DISEÑO DEL SISTEMA.....	17
3.1. ELABORACIÓN DE TARJETAS CRC	17
3.1.1 Tarjeta CRC: Usuario:	17
3.1.2 Tarjeta CRC: RegistroHabit:	18
3.1.3 Tarjeta CRC: ControladorPrediccion:	19
3.1.4 Tarjeta CRC: ModeloML:	19

3.1.5 Tarjeta CRC: GeneradorSugerencias:	20
3.1.6 Tarjeta CRC: ReporteRendimiento:	20
3.1.7 Diagrama de Colaboración entre Clases:	21
3.2. DISEÑO DE LA BASE DE DATOS.....	22
3.2.1 Modelo Conceptual:.....	22
3.2.1.1. Entidades identificadas:.....	22
3.2.1.2 Relaciones entre entidades	23
3.2.1.3 Diagrama Entidad-Relación conceptual.....	24
3.2.2 Modelo Lógico:	25
3.2.2.1. Transformación a modelo relacional:	25
3.2.2.2 Diagrama ER lógico:.....	25
3.2.2.3 Normalización (3FN):.....	27
3.2.3 Modelo Físico:	27
3.2.3.1 Diagrama ER físico (SQLite):	28
3.2.3.2. Script SQL de creación:.....	29
3.2.3.3 Restricciones y validaciones:.....	31
3.3. ELABORACIÓN DE FORMULARIOS Y REPORTES	32
3.3.1. Diseño de Formularios:	32
3.3.1.1. Validaciones de los formularios:	34
3.3.2. Diseño de Reportes:	35
3.3.3. Navegación e interfaz del sistema	36
3.3.4. Prototipos y criterios de diseño.....	37

SISTEMA INTELIGENTE DE PREDICCIÓN DE RENDIMIENTO ACADÉMICO, UNA PLATAFORMA QUE USA IA PARA ANALIZAR HÁBITOS DE ESTUDIO Y PREDECIR RESULTADOS, SUGIRIENDO MEJORAS PERSONALIZADAS

1. INVESTIGACION PRELIMINAR

Según Kendall y Kendall (2011), la investigación preliminar constituye una etapa inicial del desarrollo de sistemas en la que se analiza una solicitud, se aclara el problema que se pretende resolver y se determina si la propuesta es viable de acuerdo con los recursos técnicos, operativos y económicos disponibles.

En el presente proyecto, esta etapa permite delimitar la propuesta de SIPRA, un sistema inteligente que analizará información relacionada con los hábitos de estudio y otros factores académicos para estimar el rendimiento del estudiante y generar recomendaciones personalizadas. El sistema será desarrollado y probado inicialmente en un entorno local; posteriormente, se desplegará en la nube para que los usuarios puedan registrarse y acceder a la plataforma mediante internet.

1.1. ACLARACIÓN DE LA SOLICITUD

La aclaración de la solicitud permite convertir una idea general en una propuesta de sistema concreta. Para ello, se identifican el problema, los usuarios, las funciones principales, los datos que serán procesados y las condiciones de operación de la solución.

La solicitud del proyecto consiste en desarrollar una aplicación web que utilice técnicas de aprendizaje automático para analizar hábitos de estudio, predecir el nivel de rendimiento académico y proporcionar sugerencias que ayuden al usuario a mejorar sus resultados. Durante la fase de desarrollo, la aplicación funcionará localmente para

facilitar la programación y las pruebas. En la fase de operación, será publicada en la nube para permitir el registro y acceso de usuarios desde un navegador web.

Pregunta	Respuesta
¿Qué tipo de software se necesita?	Una aplicación web accesible desde un navegador.
¿Quiénes serán los usuarios?	Estudiantes, autodidactas y otras personas interesadas en conocer y mejorar su rendimiento académico.
¿Cómo accederán al sistema?	Mediante un registro y un sistema de inicio de sesión con correo electrónico y contraseña.
¿Dónde funcionará el sistema?	Durante el desarrollo funcionará en un entorno local. Una vez concluido, será desplegado en la nube para permitir el acceso mediante internet.
¿Qué tecnología principal utilizará?	Un modelo de aprendizaje automático para realizar predicciones y generar recomendaciones basadas en los datos ingresados.
¿Qué tipo de datos manejará?	Hábitos de estudio, horas de estudio, descanso, asistencia, participación, calificaciones previas y otros factores que se definan para el modelo.
¿El sistema generará únicamente predicciones?	No. También generará recomendaciones personalizadas relacionadas con los hábitos y las condiciones académicas del usuario.
¿Se integrará con sistemas externos?	En la primera versión no se contempla una integración obligatoria con sistemas externos. La plataforma contará con su propio registro de usuarios y almacenamiento de información.

A partir de las respuestas anteriores, se establece que SIPRA será una plataforma web con autenticación de usuarios, análisis de datos académicos y de hábitos, predicción mediante aprendizaje automático y generación de recomendaciones. El desarrollo local se considera una condición temporal de construcción y prueba, mientras que el funcionamiento previsto para los usuarios será mediante un servicio de nube.

1.2. ESTUDIO DE FACTIBILIDAD

El estudio de factibilidad permite analizar si el sistema puede desarrollarse, implementarse y utilizarse de manera satisfactoria. Para SIPRA se consideran la disponibilidad de tecnologías, la forma de uso por parte de los usuarios, los costos asociados al desarrollo y despliegue, y los riesgos que podrían afectar el proyecto.

1.2.1 Factibilidad Técnica:

La factibilidad técnica analiza si existen las herramientas, tecnologías y conocimientos necesarios para construir y poner en funcionamiento el sistema. SIPRA puede desarrollarse utilizando tecnologías web para la interfaz, un servidor backend para procesar las solicitudes, una base de datos para almacenar la información y un modelo de aprendizaje automático para realizar las predicciones. El sistema se probará inicialmente en un entorno local y luego se desplegará en un servicio de nube.

Aspecto	Análisis
Tecnologías necesarias	Front-end: HTML, CSS y JavaScript, con la posibilidad de utilizar React o Vue.js. Back-end: Python con Flask o FastAPI. Base de datos: SQLite durante el desarrollo y una alternativa administrada o compatible con el servicio de nube para producción.
Modelo de aprendizaje automático	Se podrán evaluar algoritmos como regresión logística o lineal, árboles de decisión, Random Forest u otros modelos adecuados según el tipo de predicción. La selección final dependerá de las pruebas realizadas con el conjunto de datos.
Datos para el modelo	Se podrán utilizar datasets públicos, datos sintéticos controlados y, cuando corresponda, datos recopilados mediante el sistema. Las variables utilizadas deberán coincidir con los campos registrados por la aplicación.

Desarrollo local	Durante la construcción se utilizará un servidor local, una base de datos de prueba y herramientas de programación que permitan verificar la funcionalidad antes del despliegue.
Despliegue en la nube	La versión operativa se alojará en un servicio de nube que permita publicar el backend, la interfaz web y la base de datos, además de gestionar el acceso de usuarios.
Experiencia del equipo	El desarrollo requiere conocimientos de programación web, bases de datos, seguridad básica y aprendizaje automático. El proyecto es alcanzable para un equipo pequeño si se delimita adecuadamente el alcance.

La factibilidad técnica se considera alta, debido a que existen herramientas de código abierto, servicios de nube con planes iniciales accesibles y bibliotecas especializadas para el desarrollo web y el aprendizaje automático. Sin embargo, será necesario definir el proveedor de nube, la configuración de la base de datos y los mecanismos de seguridad antes de la publicación.

1.2.2 Factibilidad Operativa:

La factibilidad operativa determina si los usuarios podrán utilizar el sistema de forma práctica y si la solución responde a una necesidad concreta. SIPRA estará dirigido principalmente a estudiantes y personas interesadas en conocer sus hábitos y mejorar su rendimiento académico. Al funcionar en la nube, el usuario no tendrá que instalar el sistema; solamente necesitará un navegador y una conexión a internet.

Aspecto	Análisis
Usuario final	Estudiantes y otras personas con conocimientos básicos de navegación web.
Acceso	El usuario podrá registrarse e iniciar sesión desde la página web utilizando sus credenciales.

Facilidad de uso	La interfaz deberá presentar formularios claros, instrucciones comprensibles y resultados fáciles de interpretar.
Disponibilidad	El acceso dependerá de la disponibilidad del servicio de nube y de la conexión a internet del usuario.

La factibilidad operativa se considera alta, siempre que se diseñe una interfaz sencilla y se proporcionen instrucciones claras. También será necesario informar al usuario sobre el propósito de la predicción y aclarar que los resultados son estimaciones de apoyo, no evaluaciones académicas oficiales.

1.2.3 Factibilidad Económica:

La factibilidad económica analiza los recursos financieros necesarios para desarrollar, publicar y mantener el sistema. El desarrollo puede realizarse utilizando herramientas gratuitas y bibliotecas de código abierto; no obstante, el funcionamiento en la nube puede generar costos por alojamiento, almacenamiento, base de datos, dominio y consumo de recursos.

Concepto	Costo estimado (USD / tiempo)
Software y herramientas	Se pueden utilizar herramientas gratuitas o de código abierto, como Python, bibliotecas de aprendizaje automático, editores de código y sistemas de control de versiones.
Hardware para desarrollo	Se requiere una computadora capaz de ejecutar el entorno de programación y realizar las pruebas del modelo.
Servicio de nube	Puede comenzar con un plan gratuito o de prueba, pero se deben revisar los límites de almacenamiento, procesamiento y transferencia de datos.
Base de datos	Durante el desarrollo puede utilizarse una base de datos local. En producción se deberá considerar el costo y la configuración del servicio de base de datos en la nube.

Dominio y certificado	El dominio puede ser opcional en la etapa inicial. Para una plataforma pública se recomienda utilizar HTTPS y un dominio apropiado.
-----------------------	---

La factibilidad económica se considera media-alta para una primera versión, debido a que el desarrollo puede realizarse con recursos gratuitos o de bajo costo. Sin embargo, no se debe afirmar que el costo total será necesariamente de cero, porque el despliegue permanente en la nube puede generar gastos según el proveedor y el número de usuarios.

1.2.4 Riesgos identificados y mitigación:

La identificación de riesgos permite anticipar situaciones que podrían afectar el desarrollo, la seguridad, la disponibilidad o la precisión del sistema. En SIPRA se consideran riesgos relacionados con el modelo de aprendizaje automático, la información de los usuarios, el servicio de nube y la facilidad de uso de la plataforma.

Riesgo	Mitigación
El modelo no alcanza la precisión esperada	Evaluar varios algoritmos, preparar adecuadamente los datos, utilizar validación cruzada y reportar métricas como exactitud, precisión, recall y F1-score.
Las variables del dataset no coinciden con las del sistema	Definir previamente las variables de entrada y utilizar un conjunto de datos compatible o realizar un proceso documentado de transformación.
Pérdida de información	Realizar copias de seguridad periódicas de la base de datos y establecer un procedimiento de recuperación.
Acceso no autorizado a las cuentas	Utilizar contraseñas protegidas mediante algoritmos adecuados de hashing, sesiones seguras y validación de credenciales.

Interrupción del servicio de nube	Elegir un proveedor confiable, monitorear el servicio y conservar copias de respaldo que permitan recuperar la información.
Costos inesperados de la nube	Controlar el consumo de recursos, configurar límites y revisar periódicamente el plan contratado.
Dificultad para utilizar el sistema	Diseñar una interfaz intuitiva, incorporar instrucciones y realizar pruebas con usuarios antes de la publicación.
Interpretación incorrecta de la predicción	Mostrar una explicación sencilla del resultado e indicar que la predicción es una estimación de apoyo y no una calificación oficial.
Uso inadecuado de datos personales	Solicitar únicamente la información necesaria, protegerla y comunicar al usuario el propósito de su recopilación.

2. DETERMINACIÓN DE REQUERIMIENTOS

Según Sommerville (2011), la ingeniería de requisitos comprende las actividades destinadas a descubrir, analizar, documentar y comprobar los servicios que debe proporcionar un sistema, así como las restricciones que deben cumplirse durante su funcionamiento.

En este punto se determinan los requisitos del proyecto, considerando las necesidades de los estudiantes, las funciones de la plataforma web, el procesamiento de datos mediante aprendizaje automático y las condiciones necesarias para su despliegue en la nube.

2.1. RECOPIACIÓN DE INFORMACIÓN

La recopilación de información permite obtener los datos necesarios para comprender el problema, identificar las expectativas de los usuarios y establecer las funciones que

deberá cumplir el sistema. En esta etapa se revisan fuentes académicas y técnicas, sistemas similares, datasets educativos y las necesidades específicas del proyecto.

Para este sistema se analizaron referencias relacionadas con la predicción del rendimiento académico, plataformas educativas y modelos de aprendizaje automático. También se definieron las condiciones de uso de la plataforma: el desarrollo y las pruebas iniciales se realizarán localmente, mientras que la versión operativa se desplegará en la nube para permitir el registro y acceso de usuarios mediante internet.

2.1.1. Técnicas Utilizadas:

Las técnicas de recopilación se seleccionaron de acuerdo con el alcance del proyecto. Su finalidad fue identificar las funciones principales, los datos que ingresará el usuario, las condiciones de seguridad y las características necesarias para presentar una predicción comprensible.

Técnica	Aplicación	Resultado obtenido
Revisión documental	Se revisaron libros, artículos y documentación técnica sobre ingeniería de requisitos, minería de datos educativos, aprendizaje automático y desarrollo de aplicaciones web.	Se identificaron conceptos, métodos de análisis, métricas de evaluación y prácticas para documentar requisitos.
Análisis de sistemas similares	Se analizaron plataformas y proyectos relacionados con la predicción del rendimiento académico, formularios de hábitos y paneles de resultados.	Se identificaron funciones como registro, autenticación, captura de datos, predicción, historial y recomendaciones.
Entrevista de aclaración	Se realizó una consulta con el solicitante del proyecto para	Se estableció que SIPRA será una aplicación web, se

	definir el alcance, usuarios, tipo de plataforma, variables de entrada y forma de despliegue.	desarrollará inicialmente de forma local y se publicará posteriormente en la nube.
Observación de interfaces	Se revisaron los flujos de uso y la organización de interfaces de plataformas educativas existentes.	Se determinó la necesidad de utilizar formularios simples, mensajes claros y resultados fáciles de interpretar.
Tormenta de ideas	Se propusieron y organizaron funcionalidades relacionadas con usuarios, hábitos, predicción, recomendaciones, reportes y administración.	Se definieron los módulos principales y se identificaron funciones que pueden quedar para versiones futuras.

2.1.2. Fuentes de Información Consultadas:

Las fuentes consultadas se organizaron en fuentes académicas, técnicas y de datos. Su revisión permitió comparar enfoques, seleccionar posibles variables de entrada y establecer una base para el diseño de la solución.

Fuente	Tipo	Información obtenida y aplicación
Sommerville, Ingeniería de software	Fuente académica	Conceptos de ingeniería de requisitos, especificación, validación y gestión de requisitos.
Romero y Ventura, minería de datos educativos y analítica del aprendizaje	Fuente académica	Conceptos relacionados con el análisis de datos educativos, descubrimiento de patrones y apoyo a la toma de decisiones.
UCI Student Performance Dataset	Dataset público	Variables académicas, sociales y personales que pueden servir como referencia para construir o evaluar un modelo.
Proyectos de predicción del	Proyectos de referencia	Arquitecturas, formularios, algoritmos y formas de presentar resultados.

rendimiento académico		
Documentación de Python, Flask/FastAPI y bibliotecas de aprendizaje automático	Fuente técnica	Configuración del backend, implementación de servicios web, procesamiento de datos y carga del modelo.
Documentación del proveedor de nube	Fuente técnica	Requisitos de despliegue, variables de entorno, almacenamiento, base de datos, seguridad y límites de consumo.

La selección definitiva del dataset deberá realizarse verificando que sus variables sean compatibles con las entradas que solicitará SIPRA. Si un dataset público no contiene todas las variables necesarias, se deberá documentar la adaptación, utilizar datos sintéticos controlados o recopilar nuevos datos mediante un instrumento definido. No se deberá afirmar que una fuente es suficiente sin comprobar su cantidad de registros, variable objetivo, calidad, licencia y compatibilidad con el modelo.

2.2. ANÁLISIS DE LA INFORMACIÓN

El análisis de la información consiste en organizar, revisar y transformar los datos recopilados en requisitos claros y comprobables. En esta etapa se eliminan ambigüedades, se identifican posibles contradicciones, se agrupan las necesidades por función y calidad, y se determina cuáles son indispensables para la primera versión del sistema.

Para el Sistema Inteligente de Predicción del Rendimiento Académico, el análisis se realizó considerando el flujo principal del usuario: registrarse, iniciar sesión, ingresar sus datos, solicitar una predicción, recibir recomendaciones y consultar sus resultados. También se consideran los requisitos técnicos relacionados con seguridad, rendimiento, disponibilidad, usabilidad y despliegue en la nube.

2.2.1 Clasificación de Requerimientos:

Los requisitos funcionales describen las acciones que el software debe realizar. Los requerimientos no funcionales describen las condiciones de calidad, seguridad, rendimiento y operación que debe cumplir la plataforma. Para evitar duplicaciones, se utilizará la codificación FR para requisitos funcionales y NFR para requisitos no funcionales.

2.2.1.1. Requerimientos Funcionales:

ID	Nombre	Descripción	Prioridad
FR-01	Registro de usuario	El sistema debe permitir crear una cuenta mediante nombre, correo electrónico y contraseña.	Alta
FR-02	Inicio de sesión	El sistema debe autenticar al usuario mediante sus credenciales y permitir el acceso únicamente si son válidas.	Alta
FR-03	Cierre de sesión	El sistema debe permitir finalizar la sesión activa del usuario.	Media
FR-04	Gestión del perfil	El sistema debe permitir consultar y actualizar la información básica del perfil del usuario.	Media
FR-05	Registro de hábitos y datos académicos	El sistema debe presentar un formulario para registrar horas de estudio, horas de sueño, asistencia, participación, uso de recursos digitales, calificaciones previas y las demás variables definidas para el modelo.	Alta
FR-06	Validación de datos	El sistema debe validar que los campos obligatorios tengan formato correcto y que los valores se encuentren dentro de los rangos establecidos.	Alta

FR-07	Predicción del rendimiento	El sistema debe procesar los datos mediante el modelo de aprendizaje automático y mostrar un nivel de rendimiento académico definido para el proyecto.	Alta
FR-08	Recomendaciones personalizadas	El sistema debe generar recomendaciones relacionadas con los hábitos y factores que puedan contribuir a mejorar el rendimiento.	Alta
FR-09	Historial de resultados	El sistema debe almacenar y permitir consultar las predicciones y recomendaciones anteriores del usuario.	Media
FR-10	Visualización del progreso	El sistema debería mostrar la evolución de los resultados mediante tablas o gráficos cuando exista información suficiente.	Baja
FR-11	Administración básica	El sistema debería permitir al usuario administrador consultar usuarios y supervisar información general del funcionamiento de la plataforma.	Media
FR-12	Exportación de resultados	El sistema podrá permitir la exportación de resultados en PDF o CSV en una versión posterior.	Baja

2.2.1.2. Requerimientos No Funcionales:

ID	Categoría	Especificación	Prioridad
NFR-01	Despliegue	La versión operativa deberá estar disponible en la nube mediante una dirección web; el desarrollo y las pruebas iniciales se realizarán localmente.	Alta
NFR-02	Compatibilidad	La interfaz deberá funcionar en las versiones recientes de Chrome, Firefox, Edge y otros	Media

		navegadores compatibles con los estándares web.	
NFR-03	Evaluación del modelo	El modelo deberá evaluarse con un conjunto de prueba y tendrá como meta alcanzar una exactitud mínima del 75 %, sin presentar este valor como garantía antes de las pruebas.	Alta
NFR-04	Tiempo de predicción	El sistema debería generar y mostrar una predicción en un máximo de cinco segundos después del envío de un formulario válido.	Alta
NFR-05	Tiempo de carga	Las páginas principales deberían cargar en un máximo de tres segundos bajo condiciones normales de conexión y disponibilidad del servicio.	Media
NFR-06	Seguridad de credenciales	Las contraseñas deberán almacenarse mediante un algoritmo de hashing apropiado, como bcrypt o Argon2id; nunca se almacenarán en texto plano.	Alta
NFR-07	Protección de datos	El sistema deberá limitar la recopilación a los datos necesarios, proteger la comunicación mediante HTTPS y controlar el acceso a la información.	Alta
NFR-08	Disponibilidad y respaldo	La información deberá contar con copias de seguridad periódicas y un procedimiento básico de recuperación.	Media
NFR-09	Usabilidad	La interfaz deberá ser clara y comprensible para personas con conocimientos básicos de navegación web.	Alta
NFR-10	Mantenibilidad	La arquitectura deberá permitir actualizar el modelo y agregar nuevas variables sin rediseñar completamente la aplicación.	Media

NFR-11	Sesión	La sesión deberá expirar después de un periodo definido de inactividad y deberá impedir el acceso no autorizado.	Media
--------	--------	--	-------

2.2.2. Priorización de Requerimientos (MoSCoW):

La priorización MoSCoW permite establecer qué requisitos deben desarrollarse primero y cuáles pueden posponerse. Esta clasificación ayuda a delimitar el producto mínimo viable y evita que las funciones secundarias desplacen las funciones esenciales del Sistema.

Prioridad	Requisitos	Justificación
Must Have (obligatorio)	FR-01, FR-02, FR-05, FR-06, FR-07, FR-08; NFR-01, NFR-03, NFR-04, NFR-06, NFR-07, NFR-09	Representan el núcleo del sistema: autenticación, registro de datos, validación, predicción, recomendaciones, seguridad y funcionamiento web.
Should Have (debería tener)	FR-03, FR-04, FR-09, FR-11; NFR-02, NFR-05, NFR-08, NFR-11	Son importantes para mejorar la experiencia, la administración y la confiabilidad, pero pueden implementarse después de las funciones centrales.
Could Have (podría tener)	FR-10; NFR-10	Aportan valor adicional y permiten ampliar la plataforma, pero no son indispensables para la primera versión funcional.
Won't Have for the MVP (no se incluirá inicialmente)	FR-12 y otras integraciones externas no definidas	Se posponen para una versión futura con el fin de mantener controlado el alcance inicial.

La validación de datos se considera obligatoria porque una entrada incorrecta puede afectar la predicción y generar recomendaciones inadecuadas. La exportación de resultados se mantiene fuera del producto mínimo viable, aunque puede conservarse en el diseño como una función futura.

2.2.3. Análisis de Consistencia y Completitud:

La consistencia y la completitud se revisan comparando los requerimientos entre sí y verificando que puedan relacionarse con los módulos, la base de datos, los casos de uso y las pruebas del sistema. Un requisito se considera adecuado cuando es claro, necesario, verificable y compatible con el alcance del proyecto.

Criterio	Resultado de la revisión	Acción requerida
Consistencia interna	Requiere ajuste	Debe utilizarse de forma uniforme el concepto de nivel de rendimiento académico y debe eliminarse la contradicción entre funcionamiento local y despliegue en la nube.
Completitud	Parcial	Se incorporaron seguridad, respaldo, protección de datos, administración y requisitos de nube. Estos elementos deberán reflejarse también en el diseño.
Claridad	Aceptable con ajustes	Las funciones principales son comprensibles, pero deben definirse las variables exactas del formulario y las categorías de salida del modelo.
Verificabilidad	Parcial	Cada requisito debe asociarse con una prueba o criterio de aceptación medible.
Trazabilidad	Parcial	La matriz debe relacionar todos los requisitos funcionales con casos de uso y los no funcionales con módulos o componentes.

Factibilidad	Aceptable	La solución es posible, pero la precisión del modelo, el proveedor de nube y los costos deben comprobarse durante el desarrollo.
--------------	-----------	--

Como resultado de esta revisión, los requisitos se consideran una base válida para continuar con el diseño, pero no deben declararse definitivamente aprobados hasta validar las variables del modelo, la estructura de datos, la seguridad y las condiciones reales de despliegue.

2.2.4. Matriz de Trazabilidad Requerimientos-Casos de Uso:

La matriz de trazabilidad permite comprobar la relación entre cada requisito y la interacción que lo implementará. También facilita la planificación de pruebas, porque permite verificar que cada función solicitada tenga una forma concreta de demostrarse en el sistema.

Requisito	Caso de uso o componente relacionado	Prioridad
FR-01	Registrarse en el sistema	Must Have
FR-02	Iniciar sesión	Must Have
FR-03	Cerrar sesión	Should Have
FR-04	Gestionar perfil	Should Have
FR-05	Registrar hábitos y datos académicos	Must Have
FR-06	Validar datos de entrada	Must Have
FR-07	Obtener predicción de rendimiento	Must Have
FR-08	Consultar recomendaciones personalizadas	Must Have
FR-09	Consultar historial de resultados	Should Have
FR-10	Visualizar progreso académico	Could Have
FR-11	Administrar información básica	Should Have
FR-12	Exportar resultados	Won't Have for the MVP
NFR-01	Arquitectura y despliegue en la nube	Must Have

NFR-03	Evaluación del modelo de aprendizaje automático	Must Have
NFR-04	Módulo de predicción y respuesta del servidor	Must Have
NFR-06	Autenticación y almacenamiento de credenciales	Must Have
NFR-07	Protección de datos y comunicación segura	Must Have
NFR-08	Base de datos y copias de seguridad	Should Have
NFR-09	Interfaz de usuario	Must Have

3. DISEÑO DEL SISTEMA

Según Pressman y Maxim (2019), el diseño de software transforma los requisitos del sistema en una representación que permite definir la arquitectura, los componentes, las interfaces y los datos necesarios para construir la solución.

En este punto se presenta el diseño del sistema propuesto mediante tres perspectivas complementarias: la organización de las clases y sus responsabilidades, la estructura de la base de datos y la definición de los formularios y reportes que utilizará el usuario. Esta separación permite relacionar el funcionamiento interno, el almacenamiento de la información y la interacción visible de la aplicación web.

3.1. ELABORACIÓN DE TARJETAS CRC

Las tarjetas CRC —Clase, Responsabilidad y Colaborador— permiten representar las clases principales del sistema y las tareas que cada una debe realizar. En este proyecto se utilizan para organizar la lógica relacionada con usuarios, hábitos, predicciones, recomendaciones y reportes antes de implementar el código.

3.1.1 Tarjeta CRC: Usuario:

Usuario	
Responsabilidades • registrarUsuario(nombre, email, password): Permite crear una nueva cuenta de usuario en el sistema, almacenando los datos de credenciales de forma segura mediante hash de contraseña. • iniciarSesion(email, password): Autentica al usuario verificando sus credenciales contra los registros almacenados en la base de datos. • editarPerfil(datosActualizados): Permite al usuario modificar su información personal como nombre, correo electrónico y preferencias. • cerrarSesion(): Finaliza la sesión activa del usuario, invalidando el token de autenticación. • obtenerHistorialPredicciones(): Recupera la lista de predicciones anteriores asociadas al usuario desde la clase ReporteRendimiento.	Colaboradores • RegistroHabitto (para obtener datos de hábitos asociados al usuario) • ReporteRendimiento (para consultar historial de predicciones)

3.1.2 Tarjeta CRC: RegistroHabitto:

RegistroHabitto	
Responsabilidades • registrarHabitto(fecha, horasEstudio, horasSueno, asistencia, tareasEntregadas): Almacena un nuevo registro diario de hábitos de estudio en la base de datos. • calcularPromedioEstudio(): Calcula el promedio de horas de estudio del usuario en un período determinado para informar al modelo de predicción. • validarDatosHabitto(): Verifica que los datos ingresados cumplan con las restricciones de formato y rango antes de su almacenamiento. • obtenerHabittoPorPeriodo(fechaInicio, fechaFin): Recupera todos los registros de hábitos del usuario dentro de un rango de fechas específico.	Colaboradores • Usuario (para asociar el registro al usuario autenticado) • ControladorPrediccion (para enviar datos al motor de predicción)

3.1.3 Tarjeta CRC: ControladorPrediccion:

ControladorPrediccion	
Responsabilidades • ejecutarPrediccion(idRegistro): Orquesta el proceso completo de predicción, coordinando la obtención de datos de hábitos y la invocación del modelo ML. • formatearEntrada(datosHabito): Transforma los datos crudos del registro de hábitos al formato requerido por el modelo de Machine Learning (normalización y codificación). • almacenarResultado(resultado): Guarda el resultado de la predicción en la base de datos asociándolo al registro de hábitos correspondiente. • obtenerPrediccionActual(idUsuario): Recupera la predicción más reciente del usuario para su presentación en el reporte.	Colaboradores • RegistroHabito (para obtener los datos de entrada de la predicción) • ModeloML (para ejecutar la inferencia del modelo entrenado) • GeneradorSugerencias (para activar la generación de recomendaciones) • ReporteRendimiento (para almacenar y consultar resultados)

3.1.4 Tarjeta CRC: ModeloML:

ModeloML	
Responsabilidades • cargarModelo(rutaModelo): Carga el modelo de Machine Learning previamente entrenado desde el sistema de archivos local al inicio de la aplicación. • predecir(datosFormateados): Ejecuta la inferencia del modelo sobre los datos de entrada y retorna el nivel de rendimiento predicho (Alto/Medio/Bajo) con la confianza asociada. • validarPrecision(): Verifica que la precisión del modelo cargado cumpla con el umbral mínimo del 75% establecido en los requerimientos.	Colaboradores • ControladorPrediccion (recibe solicitudes de predicción y retorna resultados)

• reentrenar(datosHistoricos): Permite reentrenar el modelo con datos históricos acumulados para mejorar su precisión a lo largo del tiempo.	
---	--

3.1.5 Tarjeta CRC: GeneradorSugerencias:

GeneradorSugerencias	
Responsabilidades • generarSugerencias(nivelRendimiento, datosHabito): Crea recomendaciones personalizadas basadas en el nivel de rendimiento predicho y los hábitos específicos del estudiante. • categorizarSugerencia(): Clasifica cada sugerencia en categorías como hábitos de estudio, descanso, asistencia o entrega de tareas para organizar la presentación. • obtenerSugerenciasPorPrediccion(idPrediccion): Recupera las sugerencias asociadas a una predicción específica desde la base de datos. • priorizarSugerencias(): Ordena las sugerencias según su impacto potencial en el rendimiento académico, mostrando primero las más relevantes.	Colaboradores • ControladorPrediccion (recibe el resultado de la predicción como desencadenante) • RegistroHabito (utiliza los datos de hábitos para contextualizar las sugerencias)

3.1.6 Tarjeta CRC: ReporteRendimiento:

ReporteRendimiento	
Responsabilidades • generarReportePrediccion(idPrediccion): Construye el reporte visual de la predicción actual, incluyendo indicadores de color y nivel de rendimiento. • generarHistorial(idUsuario): Genera una tabla con el historial completo de predicciones del usuario, incluyendo fechas, hábitos y resultados.	Colaboradores • Usuario (para obtener información del perfil y asociar reportes) • ControladorPrediccion (para obtener datos de predicciones) • GeneradorSugerencias (para incluir sugerencias en los reportes)

• <code>exportarReporte(formato, datos)</code>: Permite exportar los reportes en formato PDF o CSV para su uso externo. • <code>calcularEstadisticas(idUsuario)</code>: Calcula estadísticas agregadas como promedio de rendimiento, tendencias y variación a lo largo del tiempo.	
---	--

3.1.7 Diagrama de Colaboración entre Clases:

El siguiente diagrama textual ilustra el flujo de mensajes principal entre las clases identificadas. Las flechas indican la dirección de la comunicación, es decir, qué clase invoca a otra para cumplir con sus responsabilidades. Este flujo representa el caso de uso central del sistema: un usuario registra sus hábitos de estudio, el sistema predice su rendimiento y genera sugerencias personalizadas, todo ello reflejado en reportes accesibles.

Origen	Destino	Descripción del Mensaje
Usuario	RegistroHabito	registro de hábitos diarios
RegistroHabito	ControladorPrediccion	solicitud de predicción
ControladorPrediccion	ModeloML	ejecución de inferencia ML
ControladorPrediccion	GeneradorSugerencias	activación de sugerencias
ControladorPrediccion	ReporteRendimiento	almacenamiento de resultado
GeneradorSugerencias	RegistroHabito	consulta de datos contextuales
ReporteRendimiento	Usuario	presentación de reportes
ReporteRendimiento	GeneradorSugerencias	inclusión de sugerencias

La secuencia principal del sistema sigue el flujo: el Usuario registra sus hábitos a través de RegistroHabito, el cual notifica al ControladorPrediccion para que inicie el proceso de predicción. El ControladorPrediccion delega la inferencia al ModeloML y,

con el resultado obtenido, activa el `GeneradorSugerencias` para producir recomendaciones contextuales. Finalmente, el `ReporteRendimiento` consolida toda la información y la presenta al Usuario. Este diseño modular permite que cada componente sea desarrollado, probado y mantenido de forma independiente.

3.2. DISEÑO DE LA BASE DE DATOS

El diseño de la base de datos de SIPRA sigue una metodología descendente que parte del modelo conceptual (entidades y relaciones a nivel abstracto), pasa por el modelo lógico (esquema relacional normalizado) y culmina en el modelo físico (script SQL ejecutable en SQLite). Este enfoque garantiza trazabilidad entre los requerimientos del sistema y la implementación final, asegurando integridad referencial, consistencia de datos y optimización de consultas para las operaciones de predicción de rendimiento académico.

La base de datos almacena información de usuarios (estudiantes y administradores), sus hábitos de estudio y salud, las predicciones generadas por el modelo de Machine Learning y las sugerencias personalizadas derivadas de dichas predicciones. Todas las tablas cumplen con la Tercera Forma Normal (3FN) para eliminar redundancias y anomalías de actualización.

3.2.1 Modelo Conceptual:

El modelo conceptual representa la vista abstracta del dominio de SIPRA sin considerar detalles de implementación. Se identifican las entidades principales, sus atributos clave y las relaciones que las vinculan. Este modelo sirve como contrato entre el análisis de requerimientos y el diseño lógico, y constituye la base sobre la cual se construyen las capas subsiguientes del diseño de la base de datos.

3.2.1.1. Entidades identificadas:

A partir del análisis de requerimientos y las tarjetas CRC del sistema, se han identificado cuatro entidades principales que componen el dominio de SIPRA. Cada entidad agrupa un conjunto cohesivo de atributos que reflejan un concepto del mundo real relevante para la predicción del rendimiento académico.

Entidad	Descripción	Atributos principales
Usuario	Persona que interactúa con el sistema (estudiante o administrador)	id_usuario, nombre_completo, correo, contrasena_hash, rol, fecha_registro
Registro_Habito	Captura de hábitos de estudio y salud de un estudiante	id_registro, id_usuario (FK), horas_sueno, horas_estudio_diario, asistencia_clase, participacion, uso_recursos_digitales, comidas_diarias, ejercicio_semanal, fecha_registro
Prediccion	Resultado del modelo ML sobre rendimiento esperado	id_prediccion, id_usuario (FK), id_registro (FK), rendimiento_predicho, confianza, fecha_prediccion
Sugerencia	Recomendación personalizada derivada de una predicción	id_sugerencia, id_prediccion (FK), id_usuario (FK), mensaje, categoria, prioridad, leida, fecha_sugerencia

3.2.1.2 Relaciones entre entidades

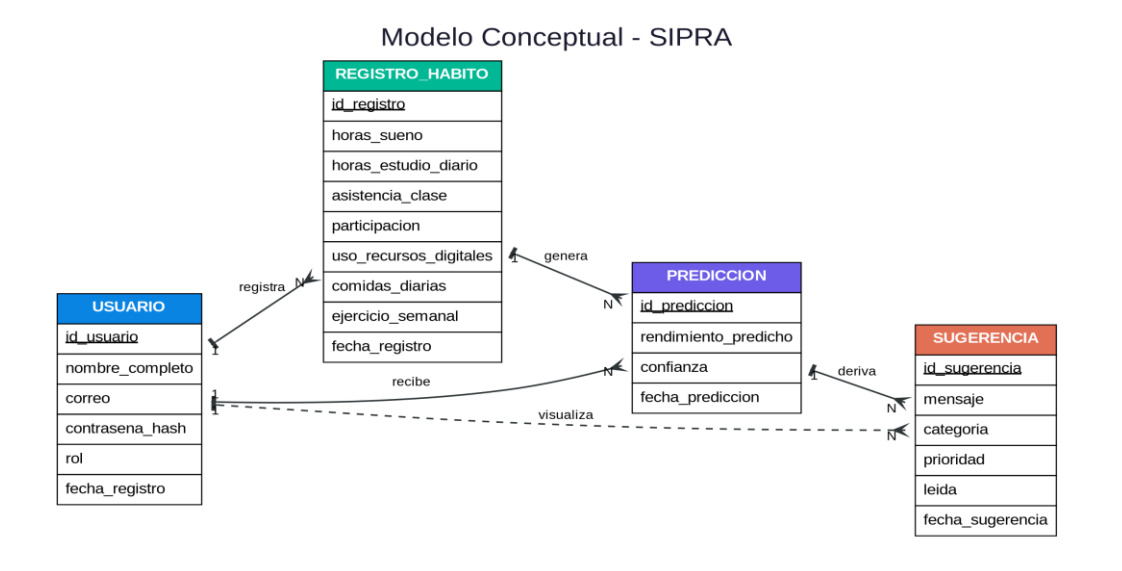
Las relaciones definen cómo interactúan las entidades entre sí en el dominio de SIPRA. Se han identificado cinco relaciones principales, cada una con su cardinalidad correspondiente. La cardinalidad determina cuántas instancias de una entidad pueden estar asociadas con instancias de otra entidad, lo cual es fundamental para la correcta traducción al modelo lógico y la definición de claves foráneas.

Relación	Verbo	Cardinalidad	Descripción
Usuario → Registro_Habito	registra	1:N	Un usuario registra múltiples hábitos a lo largo del tiempo

Usuario → Prediccion	recibe	1:N	Un usuario puede recibir múltiples predicciones
Registro_Habito → Prediccion	genera	1:N	Un registro puede generar múltiples predicciones
Prediccion → Sugerencia	deriva	1:N	Una predicción puede derivar en múltiples sugerencias
Usuario → Sugerencia	visualiza	1:N	Un usuario visualiza múltiples sugerencias

3.2.1.3 Diagrama Entidad-Relación conceptual

El siguiente diagrama muestra el modelo conceptual de SIPRA con sus entidades, atributos principales y relaciones con cardinalidad. Este diagrama utiliza la notación crow's foot (pata de cuervo) donde el símbolo "1" indica el lado uno de la relación y "N" indica el lado muchos. Las entidades están representadas como rectángulos con sus atributos listados en su interior, y las líneas de conexión incluyen el verbo que describe la semántica de cada relación.



3.2.2 Modelo Lógico:

El modelo lógico transforma las entidades y relaciones del modelo conceptual en un esquema relacional formal. Cada entidad se convierte en una tabla (relación), cada atributo se convierte en una columna, y las relaciones se implementan mediante claves foráneas (FK) que referencian a las claves primarias (PK) de las tablas relacionadas. Este modelo es independiente del motor de base de datos específico y constituye el paso intermedio antes de la implementación física.

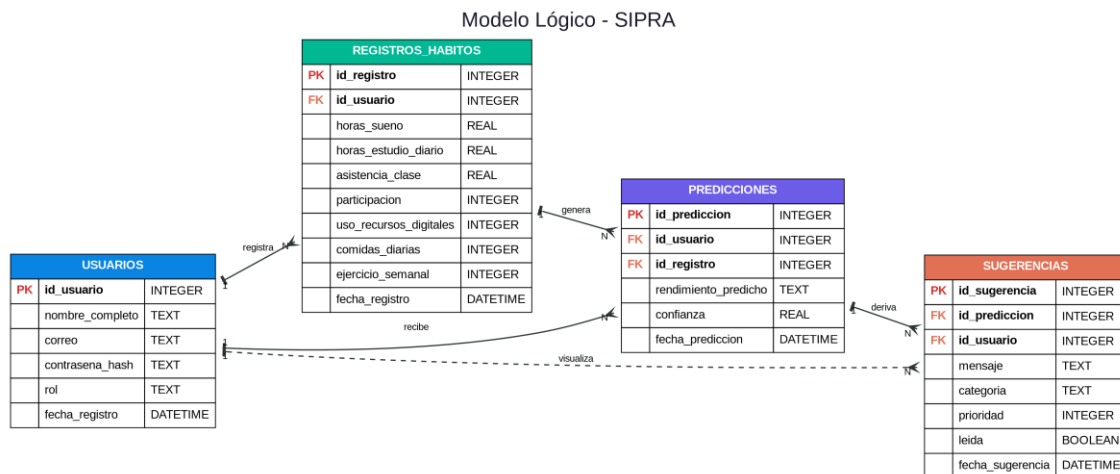
3.2.2.1. Transformación a modelo relacional:

La transformación del modelo conceptual al modelo lógico sigue las reglas estándar del modelo relacional. Las entidades se convierten directamente en tablas con sus atributos como columnas. Las relaciones 1:N se implementan agregando la clave primaria de la entidad del lado "1" como clave foránea en la entidad del lado "N". A continuación se presenta el esquema resultante de la transformación, donde se indican las claves primarias (PK) y foráneas (FK) para cada tabla.

Tabla	Clave Primaria	Claves Foráneas	Descripción
USUARIOS	id_usuario (PK)	—	Tabla base de usuarios del sistema
REGISTROS_HABITOS	id_registro (PK)	id_usuario (FK → USUARIOS)	Registros de hábitos por usuario
PREDICCIONES	id_prediccion (PK)	id_usuario (FK → USUARIOS) id_registro (FK → REGISTROS_HABITOS)	Predicciones con doble referencia
SUGERENCIAS	id_sugerencia (PK)	id_prediccion (FK → PREDICCIONES) id_usuario (FK → USUARIOS)	Sugerencias derivadas de predicciones

3.2.2.2 Diagrama ER lógico:

El siguiente diagrama muestra el modelo lógico de SIPRA con todas las tablas, sus columnas, tipos de datos genéricos, claves primarias (PK) y foráneas (FK), y las relaciones con cardinalidad entre las tablas. Este diagrama utiliza la notación crow's foot estándar de las herramientas de diseño de bases de datos, donde las flechas con "pata de cuervo" indican el lado "muchos" de la relación y la línea simple indica el lado "uno".



Lectura del diagrama:

PK (rojo): Clave primaria — identificador único de cada registro en la tabla.

FK (naranja): Clave foránea — referencia a la clave primaria de otra tabla.

Flecha continua con "pata de cuervo": Relación obligatoria (1:N).

Flecha discontinua con "pata de cuervo": Relación de visualización (1:N, no determinante).

1 / N: Cardinalidad — un usuario puede tener N registros, predicciones o sugerencias.

3.2.2.3 Normalización (3FN):

El esquema relacional de SIPRA cumple con la Tercera Forma Normal (3FN), lo que garantiza que no existen dependencias funcionales parciales ni dependencias funcionales transitivas entre los atributos. La normalización es un proceso crítico que elimina redundancias, previene anomalías de inserción, actualización y borrado, y asegura la integridad de los datos a lo largo del ciclo de vida del sistema.

Primera Forma Normal (1FN): Todos los atributos de cada tabla son atómicos (indivisibles). No existen atributos multivaluados ni atributos compuestos. Por ejemplo, en la tabla REGISTROS_HABITOS, cada hábito (horas_sueno, horas_estudio_diario, etc.) es una columna independiente con un valor único por registro, en lugar de almacenar todos los hábitos como un campo de texto concatenado.

Segunda Forma Normal (2FN): Todas las columnas no clave dependen completamente de la clave primaria, no de una parte de ella. Dado que todas las tablas utilizan claves primarias simples (de un solo atributo: id_usuario, id_registro, id_prediccion, id_sugerencia), la condición de dependencia parcial no se puede presentar, por lo que el esquema automáticamente satisface la 2FN.

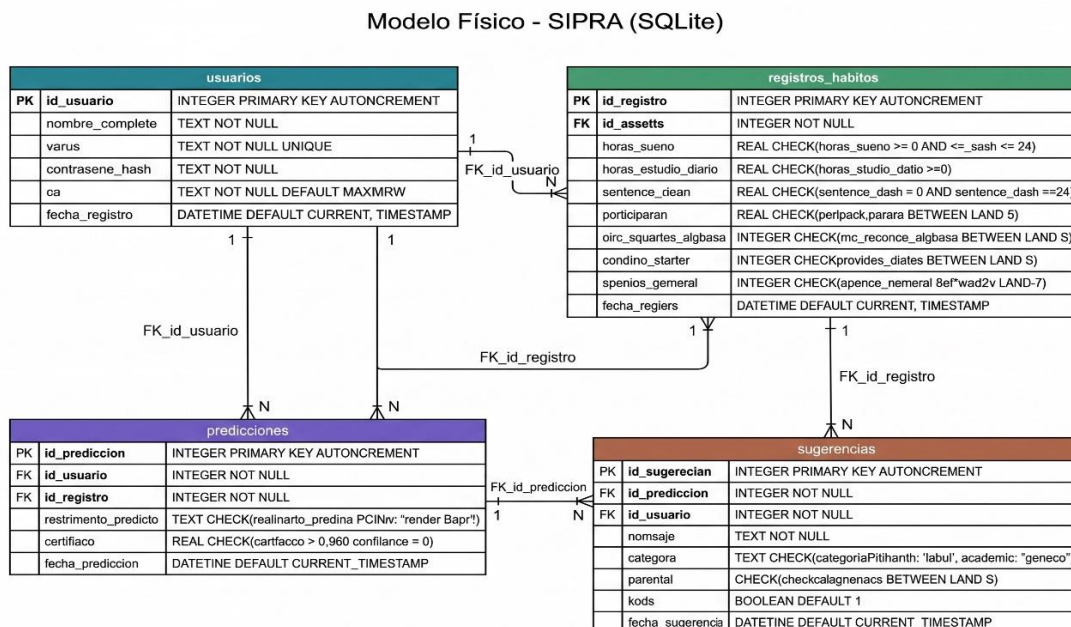
Tercera Forma Normal (3FN): No existen dependencias funcionales transitivas entre atributos no clave. Cada atributo no clave depende únicamente de la clave primaria de su tabla y no de otro atributo no clave. Por ejemplo, en la tabla PREDICCIONES, rendimiento_predicho y confianza dependen directamente de id_prediccion (la clave primaria), no de id_usuario ni de id_registro, que son claves foráneas. De igual manera, en SUGERENCIAS, mensaje, categoria y prioridad dependen de id_sugerencia, no de id_prediccion ni id_usuario.

3.2.3 Modelo Físico:

El modelo físico constituye la implementación concreta del modelo lógico en el motor de base de datos SQLite. En esta etapa se definen los tipos de datos específicos, las restricciones de integridad (NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY), los índices para optimización de consultas y las reglas de eliminación (ON DELETE CASCADE). SQLite fue seleccionado como motor por su naturaleza embebida sin servidor, lo que permite la operación local de SIPRA sin dependencia de conexión a internet ni configuración adicional.

3.2.3.1 Diagrama ER físico (SQLite):

El siguiente diagrama muestra el modelo físico de SIPRA con los tipos de datos específicos de SQLite, todas las restricciones (NOT NULL, UNIQUE, CHECK, DEFAULT, PRIMARY KEY AUTOINCREMENT) y las relaciones con las claves foráneas explícitas. Este diagrama refleja fielmente la estructura que se creará en la base de datos SQLite del sistema.



3.2.3.2. Script SQL de creación:

A continuación se presenta el script SQL completo para la creación de la base de datos SIPRA en SQLite. El script incluye la creación de tablas con todas sus restricciones, la habilitación de claves foráneas (PRAGMA foreign_keys), la creación de índices para optimizar las consultas más frecuentes y la inserción de datos iniciales para el rol administrador por defecto.

Script SQL — sipra_db.sql:

```
-- =====
-- SIPRA - Script de creación de base de datos
-- Motor: SQLite 3.x
-- Fecha: 2026-05-08
-- =====

-- Habilitar claves foráneas (requerido en SQLite)
PRAGMA foreign_keys = ON;

-- -----
-- Tabla: usuarios
-- Almacena credenciales y rol de cada usuario
-- -----
CREATE TABLE IF NOT EXISTS usuarios (
    id_usuario    INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre_completo TEXT NOT NULL,
    correo        TEXT NOT NULL UNIQUE,
    contrasena_hash TEXT NOT NULL,
    rol           TEXT NOT NULL DEFAULT 'estudiante'
                CHECK(rol IN ('estudiante', 'administrador')),
    fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- -----
-- Tabla: registros_habitos
-- Captura de hábitos de estudio y salud
-- -----
CREATE TABLE IF NOT EXISTS registros_habitos (
    id_registro    INTEGER PRIMARY KEY AUTOINCREMENT,
    id_usuario     INTEGER NOT NULL,
    horas_sueno    REAL
                CHECK(horas_sueno >= 0 AND horas_sueno <= 24),
    horas_estudio_diario REAL
                CHECK(horas_estudio_diario >= 0),
    asistencia_clase REAL
                CHECK(asistencia_clase >= 0 AND asistencia_clase <= 100),
    participacion  INTEGER
                CHECK(participacion BETWEEN 1 AND 5),
    uso_recursos_digitales INTEGER
                CHECK(uso_recursos_digitales BETWEEN 1 AND 5),
```

```

comidas_diarias      INTEGER
                     CHECK(comidas_diarias BETWEEN 1 AND 5),
ejercicio_semanal    INTEGER
                     CHECK(ejercicio_semanal BETWEEN 0 AND 7),
fecha_registro       DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
                     ON DELETE CASCADE
);

-- -----
-- Tabla: predicciones
-- Resultados del modelo ML
-- -----
CREATE TABLE IF NOT EXISTS predicciones (
  id_prediccion      INTEGER PRIMARY KEY AUTOINCREMENT,
  id_usuario         INTEGER NOT NULL,
  id_registro        INTEGER NOT NULL,
  rendimiento_predicho TEXT
                     CHECK(rendimiento_predicho IN ('Alto','Medio','Bajo')),
  confianza          REAL
                     CHECK(confianza >= 0 AND confianza <= 1),
  fecha_prediccion   DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
                     ON DELETE CASCADE,
  FOREIGN KEY (id_registro) REFERENCES registros_habitos(id_registro)
                     ON DELETE CASCADE
);

-- -----
-- Tabla: sugerencias
-- Recomendaciones personalizadas
-- -----
CREATE TABLE IF NOT EXISTS sugerencias (
  id_sugerencia      INTEGER PRIMARY KEY AUTOINCREMENT,
  id_prediccion      INTEGER NOT NULL,
  id_usuario         INTEGER NOT NULL,
  mensaje           TEXT NOT NULL,
  categoria          TEXT
                     CHECK(categoria IN ('habito','salud','academico','general')),
  prioridad          INTEGER CHECK(prioridad BETWEEN 1 AND 5),
  leida             BOOLEAN DEFAULT 0,
  fecha_sugerencia   DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_prediccion) REFERENCES predicciones(id_prediccion)
                     ON DELETE CASCADE,
  FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
                     ON DELETE CASCADE
);

-- -----
-- Índices para optimización de consultas
-- -----
CREATE INDEX IF NOT EXISTS idx_habitos_usuario
  ON registros_habitos(id_usuario);
CREATE INDEX IF NOT EXISTS idx_habitos_fecha
  ON registros_habitos(fecha_registro);
CREATE INDEX IF NOT EXISTS idx_predicciones_usuario
  ON predicciones(id_usuario);
CREATE INDEX IF NOT EXISTS idx_predicciones_registro
  ON predicciones(id_registro);
CREATE INDEX IF NOT EXISTS idx_sugerencias_prediccion

```

```

        ON sugerencias(id_prediccion);
CREATE INDEX IF NOT EXISTS idx_sugerencias_usuario
        ON sugerencias(id_usuario);
CREATE INDEX IF NOT EXISTS idx_sugerencias_leida
        ON sugerencias(leida);

-- -----
-- Datos iniciales: administrador por defecto
-- Contraseña: admin123 (hash SHA-256)
-- -----

INSERT OR IGNORE INTO usuarios
(nombre_completo, correo, contrasena_hash, rol)
VALUES
('Administrador SIPRA',
 'admin@sipra.local',
 '240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9',
 'administrador');

```

3.2.3.3 Restricciones y validaciones:

Las restricciones y validaciones implementadas en el modelo físico garantizan la integridad de los datos a nivel de base de datos, independientemente de la capa de aplicación. Estas restricciones se ejecutan automáticamente por SQLite en cada operación de inserción o actualización, proporcionando una barrera de protección adicional contra datos inconsistentes o inválidos.

Columna	Restricción	Descripción
usuarios.correo	UNIQUE	No se permiten correos duplicados en el sistema
usuarios.rol	CHECK	Solo valores: "estudiante" o "administrador"
registros_habitos.horas_sueno	CHECK	Rango válido: 0 a 24 horas
registros_habitos.asistencia_clase	CHECK	Rango válido: 0 a 100 porcentaje
registros_habitos.participacion	CHECK	Escala: 1 (muy baja) a 5 (muy alta)
registros_habitos.uso_recursos_digitales	CHECK	Escala: 1 (nada) a 5 (muy frecuente)
registros_habitos.comidas_diarias	CHECK	Rango: 1 a 5 comidas por día
registros_habitos.ejercicio_semanal	CHECK	Rango: 0 a 7 días por semana
predicciones.rendimiento_predicho	CHECK	Solo valores: "Alto", "Medio", "Bajo"

predicciones.confianza	CHECK	Rango válido: 0.0 a 1.0 (porcentaje de confianza)
sugerencias.categoria	CHECK	Solo valores: "habito", "salud", "academico", "general"
sugerencias.prioridad	CHECK	Escala: 1 (baja) a 5 (crítica)
sugerencias.leida	DEFAULT 0	Las sugerencias nuevas se crean como no leídas
Todas las FK	ON DELETE CASCADE	Al eliminar un usuario se eliminan sus registros, predicciones y sugerencias asociadas

3.3. ELABORACIÓN DE FORMULARIOS Y REPORTES

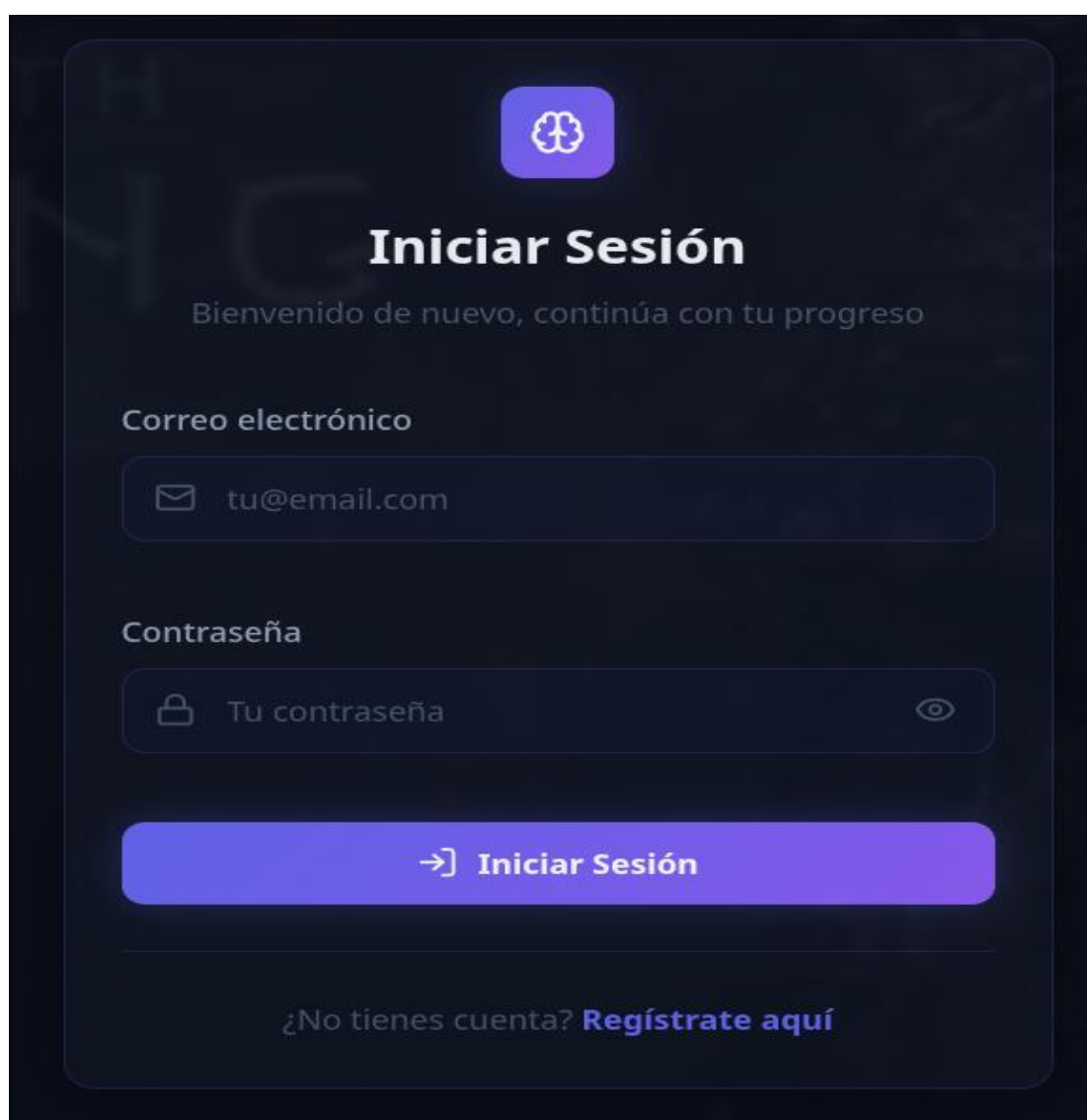
La elaboración de formularios y reportes define la interacción visible entre el usuario y la plataforma. Los formularios permiten registrar y validar información, mientras que los reportes presentan la predicción, las recomendaciones y el historial de resultados de forma comprensible. Los diseños incluidos en este apartado son mockups de referencia y podrán ajustarse durante la implementación.

3.3.1. Diseño de Formularios:

Los formularios se diseñarán con campos claros, indicaciones breves y validaciones visibles. Los campos obligatorios deberán identificarse y los mensajes de error deberán explicar cómo corregir el dato ingresado.

Formulario	Campos o elementos	Resultado esperado
Registro	Nombre, correo, contraseña, confirmación de contraseña y botón de registro.	Creación de una cuenta válida o mensaje de error.
Inicio de sesión	Correo, contraseña, botón de acceso y mensajes de autenticación.	Ingreso al panel correspondiente.

Hábitos y datos académicos	Horas de estudio, sueño, asistencia, participación, calificación previa y variables definidas para el modelo.	Datos validados y disponibles para solicitar una predicción.
Perfil	Datos personales y opción de actualizar información o contraseña.	Actualización segura del perfil.
Administración	Consulta de usuarios y estadísticas básicas, si se habilita el rol administrador.	Supervisión básica de la plataforma.



The image shows a login interface on a dark blue background. At the top center is a purple rounded square icon containing a white brain symbol. Below the icon is the title "Iniciar Sesión" in large white font. Underneath the title is the subtitle "Bienvenido de nuevo, continúa con tu progreso" in a smaller, lighter blue font. The form contains two input sections: "Correo electrónico" with a text box containing "tu@email.com" and an envelope icon, and "Contraseña" with a text box containing "Tu contraseña", a lock icon, and a toggle eye icon. A large purple button with a right-pointing arrow and the text "Iniciar Sesión" is positioned below the password field. At the bottom, a link reads "¿No tienes cuenta? **Regístrate aquí**".

3.3.1.1. Validaciones de los formularios:

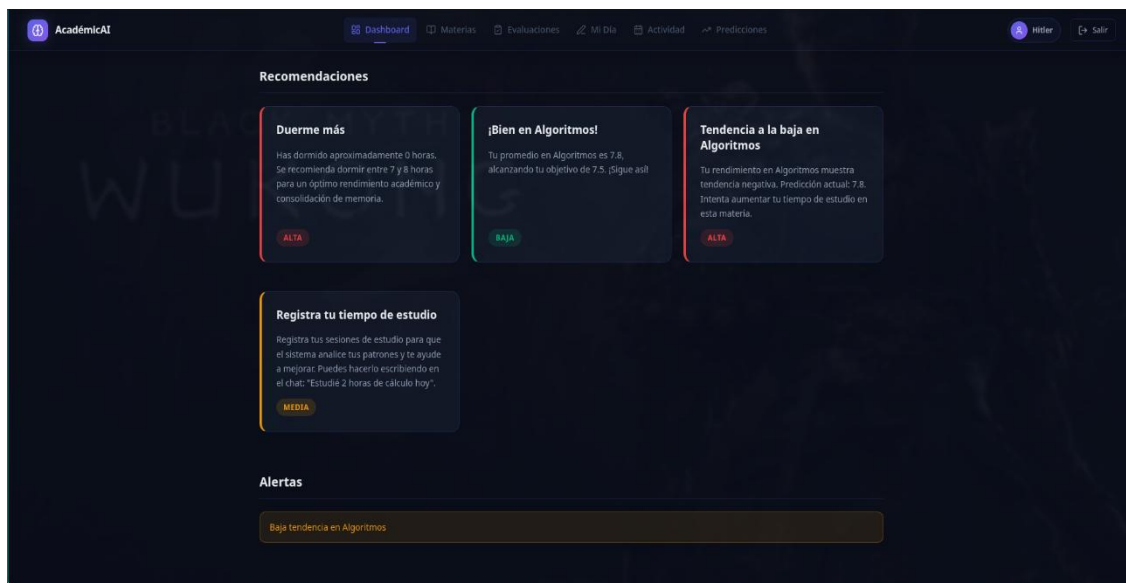
Campo o condición	Validación
Correo electrónico	Debe tener un formato válido y no estar registrado previamente.
Contraseña	Debe cumplir la política de seguridad definida y almacenarse mediante hashing.
Horas de estudio	Valor numérico mayor o igual a cero y dentro del rango establecido.
Horas de sueño	Valor entre cero y veinticuatro horas.
Asistencia	Porcentaje entre cero y cien.
Escalas de participación	Valor dentro de la escala definida, por ejemplo de uno a cinco.
Campos obligatorios	No deben enviarse vacíos.
Errores	Deben mostrarse junto al campo correspondiente con un mensaje comprensible.

3.3.2. Diseño de Reportes:

Los reportes mostrarán información procesada por el sistema de forma resumida y comprensible. El resultado no deberá presentarse como una calificación oficial, sino como una estimación que puede ayudar al usuario a identificar hábitos y áreas de mejora.

Reporte	Contenido principal	Forma de presentación
Reporte de predicción	Fecha, nivel estimado, confianza, factores considerados e interpretación.	Panel visual con indicador de nivel y explicación breve.
Reporte de recomendaciones	Mensaje, categoría, prioridad y estado de lectura.	Tarjetas o lista ordenada por prioridad.
Historial	Fechas, entradas principales, resultados y recomendaciones asociadas.	Tabla con filtros y ordenamiento.
Progreso	Evolución de resultados cuando exista suficiente información.	Gráfico de líneas o barras y resumen estadístico.
Exportación	PDF o CSV en una versión posterior.	Función futura fuera del MVP inicial.





3.3.3. Navegación e interfaz del sistema

La navegación debe permitir que el usuario complete el flujo principal sin confusiones. Después de iniciar sesión, podrá ingresar sus datos, solicitar una predicción, revisar las recomendaciones y consultar sus resultados anteriores. El menú deberá mostrar únicamente las opciones disponibles para el rol del usuario.

Paso	Pantalla o acción	Descripción
1	Inicio de sesión	El usuario introduce sus credenciales.
2	Panel principal	Se muestran accesos a hábitos, predicción, recomendaciones e historial.
3	Registro de datos	El usuario completa el formulario y corrige errores si es necesario.
4	Resultado	La plataforma procesa los datos y muestra el nivel estimado.
5	Recomendaciones	Se presentan sugerencias clasificadas y priorizadas.
6	Historial	El usuario consulta resultados de fechas anteriores.
7	Cierre de sesión	La sesión se finaliza de forma segura.

3.3.4. Prototipos y criterios de diseño

Los mockups representan una propuesta inicial de distribución visual y no constituyen todavía la interfaz definitiva. Durante la implementación se deberán validar con usuarios y ajustar según la facilidad de uso observada.

- Utilizar una jerarquía visual clara para distinguir títulos, campos, botones y resultados.
- Mostrar mensajes de éxito, advertencia y error de forma comprensible.
- Mantener consistencia en colores, tamaños, iconos y nombres de botones.
- Diseñar formularios adaptables a computadoras y dispositivos móviles.
- Evitar presentar la predicción como una certeza o como una calificación oficial.
- Permitir que el usuario vuelva a consultar sus resultados sin repetir innecesariamente los datos.